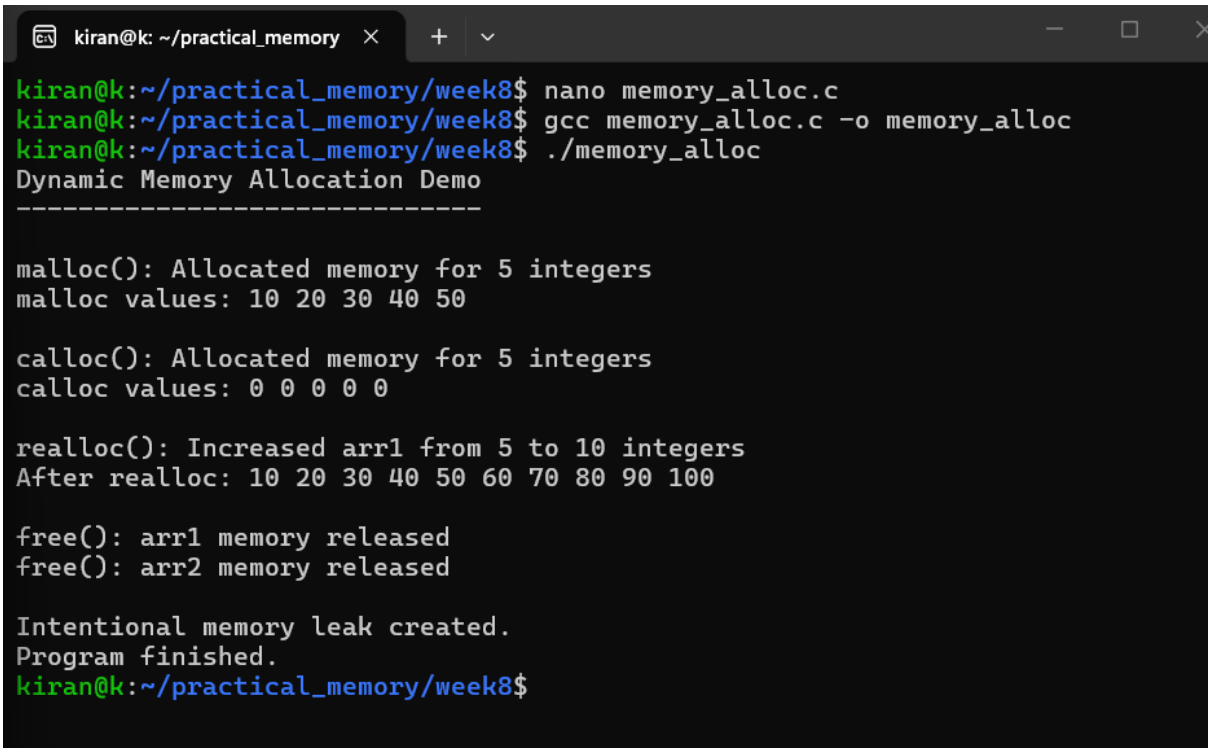
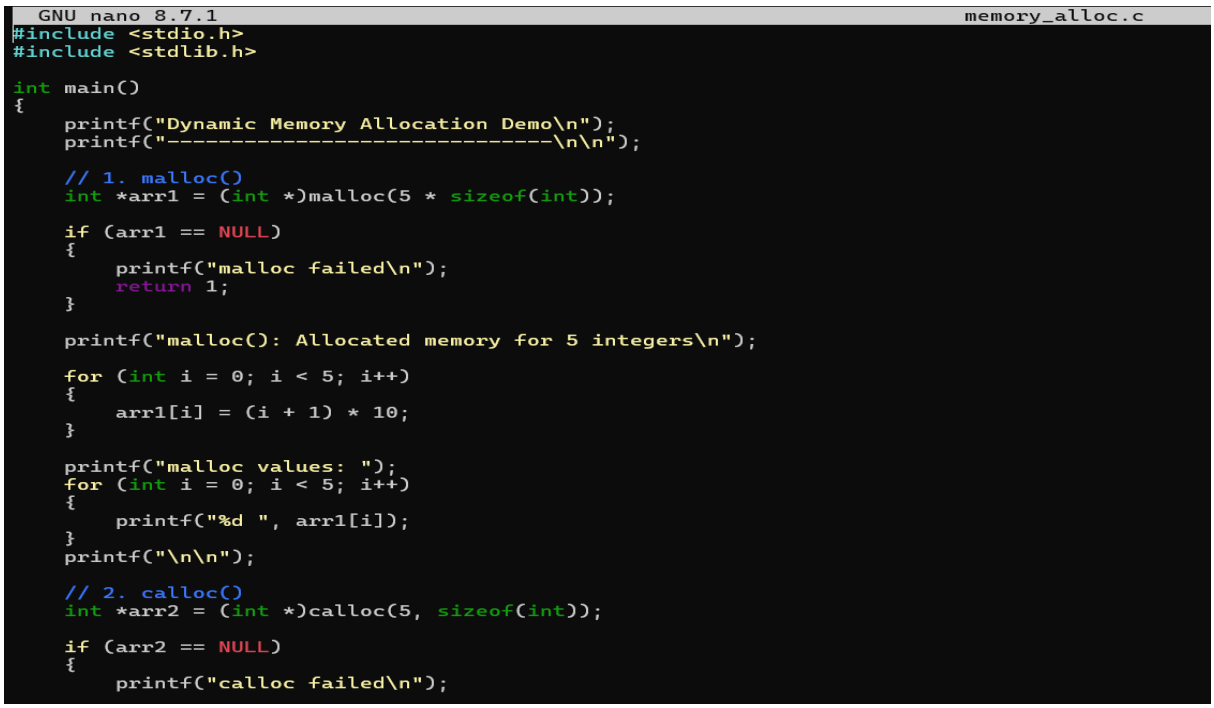
Practical Week-8

Task 1: Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.





kiran@k: ~/practical_memory

GNU nano 8.7.1 memory_alloc.c

```
        printf("calloc failed\n");
        free(arr1);
        return 1;
    }

    printf("calloc(): Allocated memory for 5 integers\n");
    printf("calloc values: ");

    for (int i = 0; i < 5; i++)
    {
        printf("%d ", arr2[i]);
    }
    printf("\n\n");

    // 3. realloc()
    arr1 = (int *)realloc(arr1, 10 * sizeof(int));

    if (arr1 == NULL)
    {
        printf("realloc failed\n");
        free(arr2);
        return 1;
    }

    printf("realloc(): Increased arr1 from 5 to 10 integers\n");

    for (int i = 5; i < 10; i++)
    {
        arr1[i] = (i + 1) * 10;
    }

    printf("After realloc: ");

    for (int i = 0; i < 10; i++)
    {
        printf("%d ", arr1[i]);
    }
```

```
    printf("\n\n");

    // 4. free()
    free(arr1);
    printf("free(): arr1 memory released\n");

    free(arr2);
    printf("free(): arr2 memory released\n");

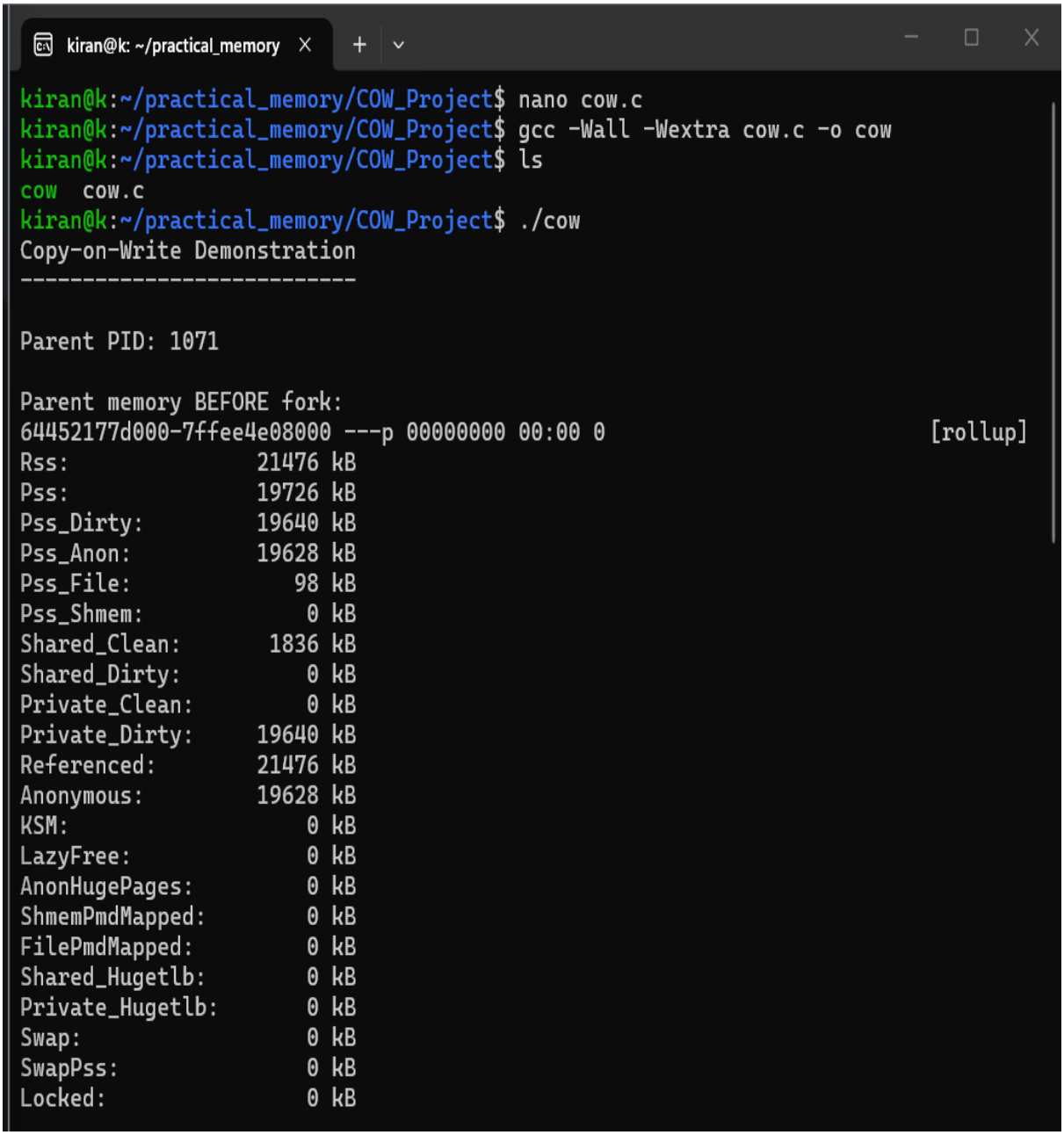
    // Intentional memory leak
    int *leak = (int *)malloc(100 * sizeof(int));

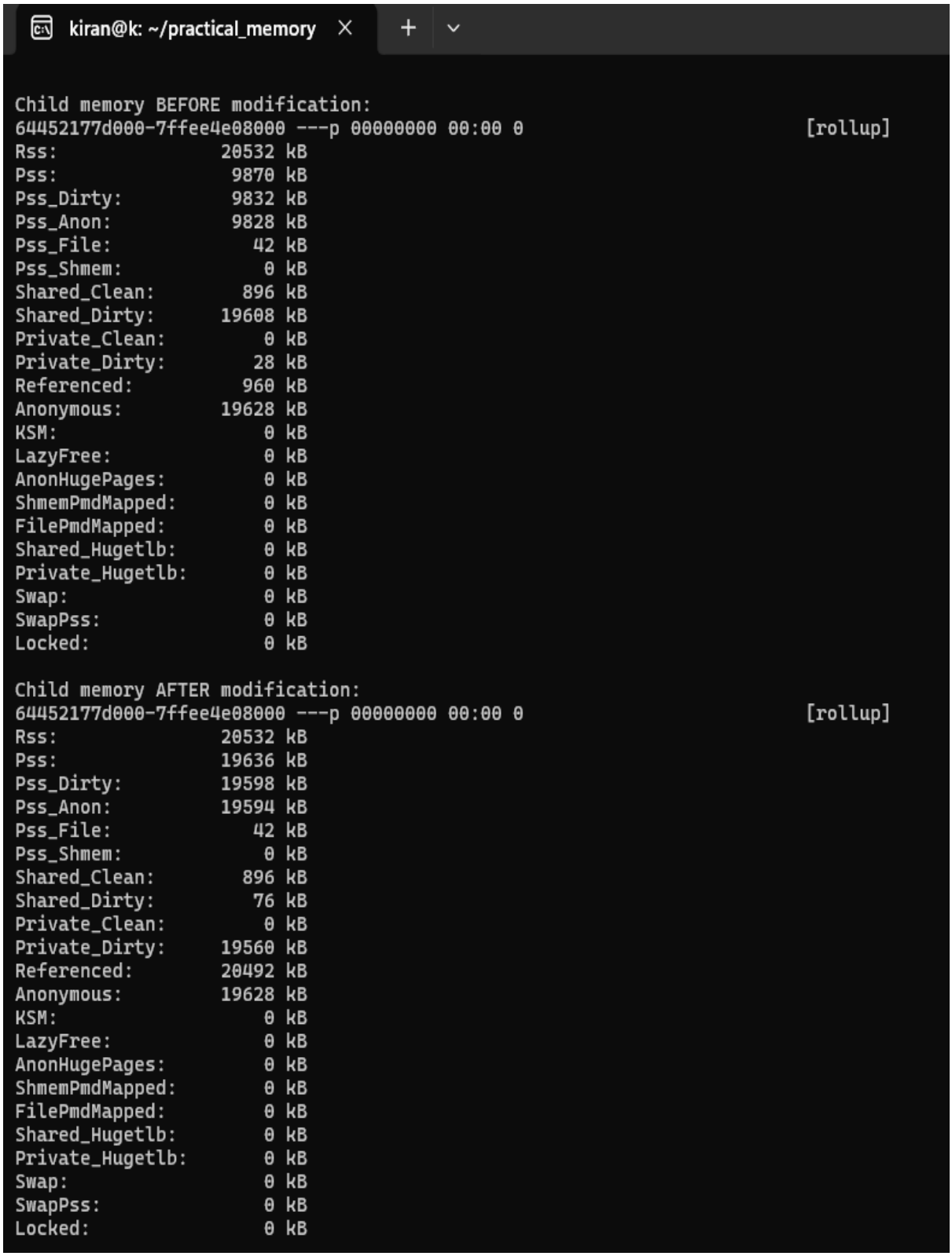
    if (leak != NULL)
    {
        printf("\nIntentional memory leak created.\n");
    }

    printf("Program finished.\n");

    return 0;
}
```

Task 2: Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.




kiran@k: ~/practical_memory
Child memory BEFORE modification:
64452177d000-7ffee4e08000 ---p 00000000 00:00 0 [rollup]
Rss: 20532 kB
Pss: 9870 kB
Pss_Dirty: 9832 kB
Pss_Anon: 9828 kB
Pss_File: 42 kB
Pss_Shmem: 0 kB
Shared_Clean: 896 kB
Shared_Dirty: 19608 kB
Private_Clean: 0 kB
Private_Dirty: 28 kB
Referenced: 960 kB
Anonymous: 19628 kB
KSM: 0 kB
LazyFree: 0 kB
AnonHugePages: 0 kB
ShmemPmdMapped: 0 kB
FilePmdMapped: 0 kB
Shared_Hugetlb: 0 kB
Private_Hugetlb: 0 kB
Swap: 0 kB
SwapPss: 0 kB
Locked: 0 kB
Child memory AFTER modification:
64452177d000-7ffee4e08000 ---p 00000000 00:00 0 [rollup]
Rss: 20532 kB
Pss: 19636 kB
Pss_Dirty: 19598 kB
Pss_Anon: 19594 kB
Pss_File: 42 kB
Pss_Shmem: 0 kB
Shared_Clean: 896 kB
Shared_Dirty: 76 kB
Private_Clean: 0 kB
Private_Dirty: 19560 kB
Referenced: 20492 kB
Anonymous: 19628 kB
KSM: 0 kB
LazyFree: 0 kB
AnonHugePages: 0 kB
ShmemPmdMapped: 0 kB
FilePmdMapped: 0 kB
Shared_Hugetlb: 0 kB
Private_Hugetlb: 0 kB
Swap: 0 kB
SwapPss: 0 kB
Locked: 0 kB

```
Parent memory AFTER child modification:
64452177d000-7ffee4e08000 ---p 00000000 00:00 0                          [rollup]
Rss:               21476 kB
Pss:               19726 kB
Pss_Dirty:         19640 kB
Pss_Anon:          19628 kB
Pss_File:             98 kB
Pss_Shmem:             0 kB
Shared_Clean:       1836 kB
Shared_Dirty:          0 kB
Private_Clean:         0 kB
Private_Dirty:     19640 kB
Referenced:        21476 kB
Anonymous:         19628 kB
KSM:                   0 kB
LazyFree:              0 kB
AnonHugePages:         0 kB
ShmemPmdMapped:        0 kB
FilePmdMapped:         0 kB
Shared_Hugetlb:        0 kB
Private_Hugetlb:       0 kB
Swap:                  0 kB
SwapPss:               0 kB
Locked:                0 kB
kiran@k:~/practical_memory/COW_Project$
```

kiran@k: ~/practical_memory

GNU nano 8.7.1 cow.c *

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void show_memory(const char *title, pid_t pid)
{
    char command[100];

    printf("\n%s\n", title);

    snprintf(command, sizeof(command),
             "cat /proc/%d/smaps_rollup", pid);

    system(command);
}

int main()
{
    printf("Copy-on-Write Demonstration\n");
    printf("---------------------------\n");

    int *data = malloc(5000000 * sizeof(int));

    if (data == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5000000; i++)
        data[i] = i;

    printf("\nParent PID: %d\n", getpid());

    show_memory("Parent memory BEFORE fork:", getpid());
```

kiran@k: ~/practical_memory

GNU nano 8.7.1 cow.c *

```c
    printf("\nParent PID: %d\n", getpid());

    show_memory("Parent memory BEFORE fork:", getpid());

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        free(data);
        return 1;
    }

    if (pid == 0)
    {
        show_memory("Child memory BEFORE modification:", getpid());

        for (int i = 0; i < 5000000; i++)
            data[i] = data[i] + 1;

        show_memory("Child memory AFTER modification:", getpid());

        free(data);
        exit(0);
    }
    else
    {
        wait(NULL);

        show_memory("Parent memory AFTER child modification:", getpid());

        free(data);
    }

    return 0;
}
```